Modern C++ (C++20) Cheatsheet

Best Practices + Patterns + Pitfalls + Interview Drill

Dense, fast-to-scan reference for high-performance, maintainable C++

January 15, 2026

Contents

1 Basics: Model, Toolchain, and Style . . . 1
- 1.1 Key rules . . . 1
- 1.2 Common pitfalls . . . 1
- 1.3 Interview angle . . . 1

2 Types, Initialization, and Value Categories . . . 2
- 2.1 Initialization styles: `{}` vs `=` vs `()` . . . 2
- 2.2 `auto`, type deduction, and `decltype` . . . 3
- 2.3 `constexpr`, `consteval`, and compile-time programming . . . 4
- 2.4 Value categories (lvalue/rvalue, prvalue/xvalue) and move . . . 4

3 Lifetime, Scope, Linkage, and the One Definition Rule . . . 5
- 3.1 Scope and lifetime . . . 5
- 3.2 Linkage, ODR, and "what goes in headers" . . . 6
- 3.3 Headers vs Modules (C++20) . . . 6

4 Pointers, References, Nullability, and Ownership . . . 7
- 4.1 References vs pointers . . . 7
- 4.2 Ownership guidance (raw pointers and smart pointers) . . . 8

5 Resource Management, RAII, and Object Semantics . . . 9
- 5.1 RAII as the backbone . . . 9
- 5.2 Rule of 0/5 and special member functions . . . 9
- 5.3 Swap and exception safety levels . . . 10

6 API Design: Parameters, Returns, Views, and Error Handling . . . 11
- 6.1 Parameter passing rules (practical and fast) . . . 11
- 6.2 Return strategies and lifetime-safe views . . . 12
- 6.3 Const-correctness, invariants, and contracts . . . 12
- 6.4 Error handling policy: exceptions vs error codes . . . 13

7 Classes, Encapsulation, and Polymorphism . . . 14
- 7.1 Concrete types vs abstract types . . . 14
- 7.2 Virtual destructors, slicing, and object layout . . . 14
- 7.3 Pimpl and compilation firewalls . . . 15

8 Templates, Concepts, and Generic Programming . . . 16
- 8.1 Template fundamentals and deduction . . . 16
- 8.2 SFINAE vs Concepts (C++20) . . . 16
- 8.3 Perfect forwarding and reference collapsing . . . 17

9 Standard Library Mastery: Containers, Algorithms, Ranges, Utilities . . . 18
- 9.1 Container selection (rules of thumb) . . . 18
- 9.2 Algorithms: prefer algorithms to loops . . . 19
- 9.3 Ranges and views (C++20) . . . 19
- 9.4 Utilities: `optional`, `variant`, `any`, `tuple`, `chrono` . . . 20

10 Concurrency (Practical): Threads, Locks, Atomics, and Async . . . 21
10.1 Threads and tasks: choosing the model . . . 21
10.2 Mutexes and lock management . . . 21
10.3 Atomics (high-level guidance only) . . . 22
10.4 `async`, futures, and waiting . . . 23
11 Performance Engineering: Cost Model and Optimization Discipline . . . 23
11.1 Cost model (what usually dominates) . . . 23
11.2 Move semantics: benefits and limits . . . 24
11.3 constexpr opportunities . . . 24
12 Architecture and System Design Guidance . . . 25
12.1 Layering, dependency direction, and boundaries . . . 25
12.2 Logging and observability . . . 26
12.3 Design checklists . . . 26
13 Pitfalls and Gotchas (High-Value UB Hotspots) . . . 27
13.1 UB hotspots . . . 27
13.2 Implicit conversions, narrowing, and overload traps . . . 27
13.3 Initialization order fiasco . . . 28
13.4 Dangling views and iterator invalidation . . . 29
13.5 Concurrency hazards . . . 29
14 Interview Drill: Explanations, Trick Questions, and Code Review Checklist . . . 30
14.1 Explain X like an interviewer . . . 30
14.2 Common trick questions (crisp answers) . . . 31
14.3 “What would you change in this code?” checklist . . . 31
1 Compact Reference Snippets . . . 31
1.1 Minimal “safe defaults” template for a value type . . . 31
1.2 RAII guard for custom resource . . . 32
1.3 Thread-safe initialization-on-first-use . . . 32

1 Basics: Model, Toolchain, and Style

1.1 Key rules

Rules of Thumb

- **Prefer expressing intent over cleverness**: readable code scales; clever code accumulates bugs.
- **Prefer standard library types and algorithms** over handwritten low-level code unless profiling proves otherwise.
- **Keep invariants explicit**: construction establishes validity; operations preserve it.
- **Write code for humans first**: structure with small functions, meaningful names, and clear ownership.
- **Treat warnings as errors** (project-wide) and run static analysis and sanitizers in CI.

1.2 Common pitfalls

Pitfalls / Gotchas

- Mixing ownership models (raw `new/delete` + smart pointers) → double delete or leaks.
- "It compiles" ≠ "it is safe": Undefined Behavior (UB) can look correct until it breaks in production.
- Over-optimizing without measurement: micro-optimizations that harm locality/clarity.

1.3 Interview angle

Interview Angle

Be ready to explain:

- "Why C++?" → control + performance + zero-cost abstractions + strong typing + rich libraries.
- "What makes modern C++ modern?" → RAII everywhere, move semantics, constexpr, generic programming with concepts, ranges, concurrency library.
- "How do you keep C++ safe?" → ownership discipline, RAII, invariants, contracts via assertions, avoid UB hotspots, tooling.

2 Types, Initialization, and Value Categories

2.1 Initialization styles: `{}` vs `=` vs `()`

Key rules

Rules of Thumb

- **Prefer brace initialization `{...}`** for uniform initialization and to avoid narrowing.
- Use `=` for copy initialization when you explicitly want implicit conversion.
- Use `()` primarily for:
  - constructor calls where braces trigger initializer-list overload ambiguity,
  - function-style casts (rare; prefer `static_cast`).
- **Prefer initializing over assigning**:

```
// Good: initializes once
std::string s = "abc";
std::vector<int> v{1,2,3};

// Avoid: default construct then assign
std::string s;
s = "abc";
```

Common pitfalls

Pitfalls / Gotchas

- **Initializer-list trap**:

```
std::vector<int> a(10, 1); // 10 elements of value 1
std::vector<int> b{10, 1}; // 2 elements: 10 and 1
```

- Brace-init blocks narrowing:

```
int x1 = 3.14;   // OK (but truncates)
int x2{3.14};    // ERROR: narrowing
```

Interview angle

Interview Angle

Explain why `{...}` is often preferred (uniformity + narrowing protection) and when `()` is necessary (initializer-list overload resolution, “most vexing parse” avoidance, etc.).

2.2 `auto`, type deduction, and `decltype`

Key rules

Rules of Thumb

- **Prefer `auto`** when:
  - the type is obvious from the RHS,
  - the exact type is verbose (iterators, lambdas),
  - you want to avoid narrowing/implicit conversions surprises via consistent deduction.
- **But keep readability**: avoid `auto` if it hides important semantics (ownership, units, signedness).
- Know the big rule: **`auto` drops top-level `const` and references** unless you spell them:

```
const int ci = 7;
auto a = ci;        // int
auto& b = ci;       // const int&
const auto c = ci;  // const int
```

- Use `decltype(expr)` when you need the exact type of an expression, especially in templates.
- Use `decltype(auto)` to preserve value category in returns (with care).

Common pitfalls

Pitfalls / Gotchas

- `auto x = {1,2,3};` deduces `std::initializer_list<int>` (not a vector).
- Dangling references with `decltype(auto)` returns if returning references to locals.

Interview angle

Interview Angle

Common question: "Difference between `auto` and `decltype(auto)`?"
Crisp answer: `auto` deduces a *value type* (drops refs/cv on the deduced type); `decltype(auto)` deduces using `decltype`, preserving reference/value category exactly.

2.3 `constexpr`, `consteval`, and compile-time programming

Key rules

Rules of Thumb

- `constexpr`: can run at compile time (if given constant inputs) *and* can run at runtime.
- `consteval`: must run at compile time; use for enforced compile-time computation.
- Prefer `constexpr` data/functions for:
  - fixed tables, configuration, parsing small literals,
  - removing runtime overhead,
  - enabling stronger compiler checking.
- Keep `constexpr` functions side-effect free (or controlled) and deterministic.

Common pitfalls

Pitfalls / Gotchas

- Assuming `constexpr` means "compile-time only" (it does not).
- Overusing template metaprogramming where `constexpr` would be simpler and clearer.

Interview angle

Interview Angle

Be ready to give an example where `consteval` prevents misuse (e.g., forcing compile-time validated format strings or bitmask construction).

2.4 Value categories (lvalue/rvalue, prvalue/xvalue) and move

Key rules

Rules of Thumb

- **Mental model:**
  - lvalue: has identity (addressable object).
  - rvalue: temporary/expiring value (can be moved-from).
- **`std::move` does not move**: it *casts* to an rvalue, enabling move operations.
- **After move:** objects must remain valid but can be in unspecified state; reassign or destroy.

Common pitfalls

Pitfalls / Gotchas

- Using moved-from objects assuming contents (UB? not UB, but unspecified; logic bugs).
- Moving from `const` objects often falls back to copy (move ctors typically require non-const rvalues).

Interview angle

Interview Angle

If asked "What is an xvalue?"
Answer: an expiring value (has identity, can be moved-from), e.g., `std::move(x)` or a function returning `T&&`.

3 Lifetime, Scope, Linkage, and the One Definition Rule

3.1 Scope and lifetime

Key rules

Rules of Thumb

- Prefer **automatic storage** (stack) objects with RAII.
- Minimize **dynamic storage duration**; if needed, express ownership with smart pointers.
- Keep lifetimes simple: **do not return references/pointers to locals**.
- Prefer **small scopes**: declare variables close to first use.

Common pitfalls

Pitfalls / Gotchas

- Dangling references to temporaries:

```
std::string_view v = std::string("tmp"); // dangling!
```

- Capturing dangling references in lambdas (especially async).

Interview angle

Interview Angle

Expect: "Explain lifetime extension of temporaries."
Key points: binding a temporary to a *const reference* in the same full-expression can extend lifetime; `string_view/span` do *not* own and thus do not extend lifetime.

3.2 Linkage, ODR, and "what goes in headers"

Key rules

Rules of Thumb

- **ODR:** an entity must have exactly one definition across the program, with limited inline/template exceptions.
- In headers:
  - declarations, inline functions, templates, constexpr/inline variables (C++17+),
  - small function bodies only when beneficial and stable.
- In source files:
  - non-inline function definitions, non-template definitions, heavy includes hidden behind pimpl.
- Prefer **include guards** or `#pragma once`.
- Prefer **internal linkage** for TU-local helpers: anonymous namespace or `static` (functions/-vars).

Common pitfalls

Pitfalls / Gotchas

- Defining non-inline functions in headers → multiple definitions at link time.
- Violating ODR with different macros/flags causing different class layouts across TUs.
- Global variables with dynamic initialization across TUs → initialization order fiasco.

Interview angle

Interview Angle

Common: "What is the initialization order fiasco and how do you fix it?"
Answer: order across translation units is unspecified; fix by function-local static initialization (construct on first use) or explicit initialization sequencing.

3.3 Headers vs Modules (C++20)

Key rules

Rules of Thumb

- Modules improve build times and reduce macro leakage; they are a **build + toolchain** feature as much as a language feature.
- Treat adoption as incremental: isolate stable libraries behind module interfaces first.
- Keep **module boundaries** aligned with architecture boundaries (components).

Common pitfalls

Pitfalls / Gotchas

- Assuming modules magically fix poor dependency structure (they don't).
- Mixing heavy macro-based headers into module interfaces can negate benefits.

Interview angle

Interview Angle

Be ready to describe **why modules matter**: less textual inclusion, better isolation, faster builds, fewer ODR/macro hazards.

4 Pointers, References, Nullability, and Ownership

4.1 References vs pointers

Key rules

Rules of Thumb

- Use **references** when you require a valid object (non-null, must refer).
- Use **pointers** when "no object" is a valid state or reseating is needed.
- Prefer `T&` / `const T&` for parameters unless ownership or nullability is part of the API.
- Prefer `gsl::not_null<T*>`-like semantics (or your own wrapper) in high-assurance codebases (if allowed).

Common pitfalls

Pitfalls / Gotchas

- Returning raw owning pointers without documented ownership transfer.
- Storing references as members when rebinding/assigning is needed (references are not reseatable).

Interview angle

Interview Angle

Question: "When would you use a pointer parameter?"
Answer: when "optional" is part of the semantics (nullable), or for C interop; otherwise prefer references or `std::optional/std::reference_wrapper`.

4.2 Ownership guidance (raw pointers and smart pointers)

Key rules

Rules of Thumb

- **Default:** no raw ownership. Express ownership in types.
- `std::unique_ptr<T>` for exclusive ownership; pass by value to transfer, by reference to observe.
- `std::shared_ptr<T>` for shared ownership *only when truly needed*; it encodes a global lifetime policy.
- `std::weak_ptr<T>` to break cycles and for non-owning observation of shared objects.
- Prefer `std::make_unique` / `std::make_shared`.

Common pitfalls

Pitfalls / Gotchas

- **"shared_ptr everywhere"** → hidden coupling, atomic refcount costs, leaks via cycles.
- Creating `shared_ptr` from `this` incorrectly (use `enable_shared_from_this` if needed).
- Multiple `shared_ptr` control blocks for the same raw pointer → double delete.

Interview angle

Interview Angle

Be ready to explain tradeoffs:

- `unique_ptr`: cheap, explicit ownership, movable.
- `shared_ptr`: shared lifetime, but overhead + subtle design implications.
- Weak references: avoid cycles, check liveness via `lock()`.

5 Resource Management, RAII, and Object Semantics

5.1 RAII as the backbone

Key rules

Rules of Thumb

- **RAII:** acquire resources in constructors; release in destructors; make invalid states unrepresentable.
- Resource = memory, file handle, socket, mutex, transaction, temporary state, timing/profiling scopes.
- Prefer scope-bound guards:

```
std::lock\_guard<std::mutex> lk(m); // unlocks on scope exit
```

Common pitfalls

Pitfalls / Gotchas

- Manual `new/delete` or `malloc/free` in application logic.
- Using RAII types but leaking ownership by `release()` without clear transfer.

Interview angle

Interview Angle

If asked "Why does RAII matter in C++?"
Answer: it gives deterministic cleanup, composes with exceptions, and makes resource safety largely automatic.

5.2 Rule of 0/5 and special member functions

Key rules

Rules of Thumb

- **Rule of 0:** if members manage resources via RAII types, define no destructor/copy/move.
- If you define any of: destructor, copy ctor, copy assign, move ctor, move assign ⇒ consider all five.
- Prefer `=default` and `=delete` explicitly to document intent.
- Prefer move operations **`noexcept`** when they truly cannot throw; this enables container optimizations.

Common pitfalls

Pitfalls / Gotchas

- Defining a destructor suppresses implicit move operations (unless explicitly defaulted).
- Forgetting self-assignment safety in copy assignment (use copy-and-swap when appropriate).

Interview angle

Interview Angle

Typical prompt: "Write a strong exception-safe assignment operator."
Crisp approach: copy-and-swap (or memberwise with strong guarantees) and explain strong/basic/no-throw guarantees.

5.3 Swap and exception safety levels

Key rules

Rules of Thumb

- Provide a `swap(T&,T&)` and/or `friend void swap(T&,T&)` when it materially helps.
- Exception safety levels:
  - **No-throw:** operation never throws.
  - **Strong:** on failure, program state unchanged.
  - **Basic:** invariants preserved, but state may change.
- Prefer strong guarantee for operations that users expect to be atomic (e.g., assignment).

Common pitfalls

Pitfalls / Gotchas

- Throwing from destructors during stack unwinding → `std::terminate`. Destructors should be **no-throw**.
- Marking `noexcept` incorrectly → termination on throw.

Interview angle

Interview Angle

"Why does `noexcept` matter for move?"
Answer: containers prefer move during reallocation only if move is non-throwing; otherwise they may copy to preserve strong guarantees.

6 API Design: Parameters, Returns, Views, and Error Handling

6.1 Parameter passing rules (practical and fast)

Key rules

Rules of Thumb

- **In-parameters (read-only):**
  - small trivially copyable types (ints, enums, iterators): pass by value.
  - larger types: pass by `const T&`.
- **Out-parameters:** prefer return values; otherwise `T&` (non-null).
- **In/out:** `T&` (document mutation).
- **Sinks (taking ownership):** take by value and `std::move` into members (enables copy elision/move).

```
class W {
  std::string s;
public:
  explicit W(std::string x) : s(std::move(x)) {} // copy or move in
};
```

- **Perfect forwarding:** only in generic code; do not expose forwarding references in normal APIs unless necessary.

Common pitfalls

Pitfalls / Gotchas

- Taking `const T&` for everything (misses move opportunities and makes ownership unclear).
- Taking `T&&` in non-template APIs as "optimization" (often harms usability).

Interview angle

Interview Angle

Prompt: "How would you design `setName(...)`?"
Good answer: take `std::string` by value and move into storage; overload for `std::string_view` only if you truly want non-owning view semantics.

6.2 Return strategies and lifetime-safe views

Key rules

Rules of Thumb

- Prefer returning objects by value; compilers optimize via RVO/NRVO and moves.
- Return references/pointers only when you return an existing object with guaranteed lifetime.
- Use `std::string_view` / `std::span<T>` for non-owning parameters when lifetime is guaranteed externally.
- Prefer `std::span<const T>` for "read-only buffer" API; `std::span<T>` for mutable.

Common pitfalls

Pitfalls / Gotchas

- Returning `string_view` to a temporary/local → dangling.
- Storing `string_view/span` as members without clear lifetime ownership.

Interview angle

Interview Angle

Be ready: "Why is returning by value OK in modern C++?"
Answer: copy elision (often mandatory in common cases), plus move semantics. Emphasize correctness and clarity.

6.3 Const-correctness, invariants, and contracts

Key rules

Rules of Thumb

- Use `const` to express immutability of observable state; it is part of the API contract.
- Maintain class invariants:
  - established in constructors,
  - preserved by all public operations,
  - enforced with assertions on boundaries (debug) and checked errors (release where needed).
- Prefer `private` data + narrow public interface; expose minimal surface area.

Common pitfalls

Pitfalls / Gotchas

- "Logical constness" vs physical constness: `mutable` is powerful but can violate thread-safety assumptions.
- Overusing getters/setters that leak representation and destroy invariants.

Interview angle

Interview Angle

Question: "What is an invariant?"
Answer: a property that must always hold for an object between public calls; it defines the valid state space.

6.4 Error handling policy: exceptions vs error codes

Key rules

Rules of Thumb

- Pick a **project-wide policy**:
  - **Exceptions** for failure that is rare and cannot be handled locally; best with RAII.
  - **Error codes** for expected/control-flow-like outcomes (parsing, validation) or ABI boundaries.
- Use exceptions to report failure, not to implement normal control flow.
- Prefer `std::optional<T>` for "maybe value", `std::variant<T,E>` or `expected`-like (if available) for rich errors.
- Never throw from destructors; if cleanup can fail, provide explicit `close()/commit()` APIs.

Common pitfalls

Pitfalls / Gotchas

- Swallowing exceptions silently (hides bugs).
- Mixing policies across layers (throwing through C ABI; returning error codes but also throwing) without clear boundaries.

Interview angle

Interview Angle

"When do you prefer exceptions?"
Answer: when a function cannot do its job and the caller cannot reasonably continue without handling; exceptions separate error path from happy path and compose with RAII.

7 Classes, Encapsulation, and Polymorphism

7.1 Concrete types vs abstract types

Key rules

Rules of Thumb

- Prefer **concrete types** (value semantics) when possible: simpler, faster, easier to reason about.
- Use **abstract types/interfaces** when you need runtime polymorphism and stable contracts across implementations.
- Prefer **composition over inheritance** unless substitutability is clean and stable.

Common pitfalls

Pitfalls / Gotchas

- Designing deep class hierarchies that encode policy decisions.
- Using inheritance for code reuse instead of substitutability.

Interview angle

Interview Angle

If asked "When is inheritance appropriate?"
Answer: when derived is-a base, you need runtime substitution, and the base provides a stable interface with a virtual destructor.

7.2 Virtual destructors, slicing, and object layout

Key rules

Rules of Thumb

- If a class is intended to be used polymorphically, give it a **virtual destructor**.
- Avoid object slicing: pass/store polymorphic objects via pointers/references (often smart pointers).

```
struct Base { virtual ~Base() = default; virtual void f() = 0; };
struct Der : Base { void f() override {} };

void g(Base b);          // slices!
void h(const Base& b);   // OK
```

- Mark overrides with `override`; mark non-overridable with `final`.
- Keep vtables in mind: virtual calls inhibit some inlining; measure before "optimizing away polymorphism."

Common pitfalls

Pitfalls / Gotchas

- Missing virtual destructor in base → UB on deleting through base pointer.
- Calling virtual functions in constructors/destructors (dispatch is to current base subobject).

Interview angle

Interview Angle

Explain vtable/vptr at a high level: dynamic dispatch via an indirection table; object typically stores a pointer to vtable; exact layout is implementation-defined.

7.3 Pimpl and compilation firewalls

Key rules

Rules of Thumb

- Use Pimpl to:
  - hide heavy includes/implementation details,
  - stabilize ABI,
  - reduce rebuild times.
- Cost: extra allocation/indirection; complexity in move/copy; debugging friction.
- Prefer Pimpl at module/component boundaries or ABI-facing libraries, not everywhere.

Common pitfalls

Pitfalls / Gotchas

- Pimpl in performance-critical inner loops.
- Forgetting to define move operations correctly with incomplete types in headers (use out-of-line definitions).

Interview angle

Interview Angle

If asked "Why Pimpl?" answer in terms of dependency control, ABI stability, and build scalability, then mention the runtime costs.

8 Templates, Concepts, and Generic Programming

8.1 Template fundamentals and deduction

Key rules

Rules of Thumb

- Templates are compile-time generators; keep interfaces minimal and constraints explicit.
- Prefer **type deduction that preserves intent**:
  - use `auto` and templates to avoid redundancy,
  - but avoid surprising decay (arrays/functions to pointers) when it matters.
- Prefer **non-member non-friend** generic algorithms when possible; keep types simple.

Common pitfalls

Pitfalls / Gotchas

- Error spew from unconstrained templates; fix with concepts/constraints and better diagnostics.
- Depending on ADL or implicit conversions accidentally (ambiguous overloads).

Interview angle

Interview Angle

Expect: "How does template instantiation work?"
Answer: compilation is per translation unit; templates are instantiated on use; definitions must be visible (hence in headers/modules).

8.2 SFINAE vs Concepts (C++20)

Key rules

Rules of Thumb

- Prefer **concepts** for expressing requirements: clearer APIs, better errors, safer overload sets.
- Use SFINAE only when:
  - you need legacy constraints,
  - you need fine-grained tricks not expressible (rare in C++20 codebases).
- Constrain templates early:

```cpp
#include <concepts>

template <std::integral I>
I gcd(I a, I b) { /*...*/ }
```

Common pitfalls

Pitfalls / Gotchas

- Overconstraining (reduces usability) or underconstraining (reintroduces cryptic errors).
- Using `requires` in ways that accidentally changes overload priority.

Interview angle

Interview Angle

"What problem do concepts solve?"
Answer: they make template requirements explicit, improve overload resolution, and yield comprehensible diagnostics.

8.3 Perfect forwarding and reference collapsing

Key rules

Rules of Thumb

- Use forwarding references only in templates:

```
template <class T>
void f(T&& x) { g(std::forward<T>(x)); }
```

- Reference collapsing rules (must-know):
  - `T& &` → `T&`
  - `T& &&` → `T&`
  - `T&& &` → `T&`
  - `T&& &&` → `T&&`
- **Move from named rvalue references requires `std::move`** (names are lvalues).

Common pitfalls

Pitfalls / Gotchas

- **Forwarding everything** in non-generic code (adds complexity; can introduce dangling).
- Double-moving: `std::move(x)` and then using `x` again as if unchanged.

Interview angle

Interview Angle

Classic: "Why is `std::forward` needed?"
Answer: it conditionally casts based on the original value category deduced for `T`, preserving lvalue vs rvalue intent.

9 Standard Library Mastery: Containers, Algorithms, Ranges, Utilities

9.1 Container selection (rules of thumb)

Key rules

Rules of Thumb

- Default container: **`std::vector`** (cache-friendly, compact, fast iteration).
- Need stable references/iterators across insert/erase in middle? Consider `list`/`forward_list` *only if necessary* (often slower due to locality).
- Need ordered keys? `map/set`. Need fast average lookup? `unordered_map/unordered_set`.
- Prefer `flat_map`-like patterns (sorted vector) when reads dominate and updates are batchy (if allowed in your environment).
- Prefer `reserve()` when size is known to avoid repeated reallocations.

Complexity and invalidation quick table

Container	Strengths	Common invalidation notes
`vector`	best iteration/locality; amortized push_back	reallocation invalidates all iterators/refs; erase invalidates after point
`deque`	stable push/pop front/back; no full reallocation	iterators/refs can invalidate on growth (implementation-dependent)
`list`	stable iterators on insert/erase elsewhere	no random access; poor locality
`map`	ordered; stable iterators (except erased)	iterators stable; slower per op (tree)
`unordered_map`	avg O(1) lookup	rehash invalidates iterators; references/pointers to elements typically stable only if not rehashed

Common pitfalls

Pitfalls / Gotchas

- Holding iterators/references across operations that invalidate them.
- Choosing `list` for "fast insert" without measuring (often slower overall).
- Using `unordered_map` without reserving when many inserts → repeated rehashing.

Interview angle

Interview Angle

Expect: "Why is `vector` usually fastest?"
Answer: contiguous storage → cache locality, fewer allocations, predictable iteration; algorithmic complexity is not the whole story.

9.2 Algorithms: prefer algorithms to loops

Key rules

Rules of Thumb

- Prefer standard algorithms (`sort, find, transform, accumulate, remove_if`) over hand-written loops.
- Algorithms express intent and enable better reuse and optimization.
- Use erase-remove idiom:

```
v.erase(std::remove\_if(v.begin(), v.end(), pred), v.end());
```

- Prefer `ranges` algorithms for composability and safer iterator handling.

Common pitfalls

Pitfalls / Gotchas

- Forgetting that `remove` does not erase; it shuffles and returns new logical end.
- Using algorithms with invalid iterators or mismatched ranges.

Interview angle

Interview Angle

If given loop code, propose `std::ranges` rewrite to show clarity:

- `ranges::sort(v);`
- `auto it = ranges::find(v, x);`

Discuss complexity and iterator invalidation.

9.3 Ranges and views (C++20)

Key rules

Rules of Thumb

- Views are **lazy** and typically non-owning; great for pipelines.

```
auto out = v | std::views::filter(pred)
             | std::views::transform(f);
```

- Prefer views to avoid intermediate allocations, but be mindful of lifetimes.
- Use `std::ranges` algorithms with views for clean composition.

Common pitfalls

Pitfalls / Gotchas

- Dangling views: view refers to a temporary container.
- Debugging complexity: laziness can hide when work happens; materialize with `to<vector>` (if available) or manual copy when needed.

Interview angle

Interview Angle

Good explanation: views are lightweight adaptors; they do not necessarily allocate; they can improve clarity and performance by composing transformations.

9.4 Utilities: `optional`, `variant`, `any`, `tuple`, `chrono`

Key rules

Rules of Thumb

- `optional<T>`: value may be absent (no error info by itself).
- `variant<A,B,...>`: closed set of alternatives (sum type); use `std::visit`.
- `any`: type-erased box; prefer variant when alternatives are known (variant is safer, faster, and explicit).
- `tuple`: use for heterogeneous returns; prefer named structs for stable APIs.
- `chrono`: use strong types for time; never pass raw integers for durations.

Common pitfalls

Pitfalls / Gotchas

- Using `any` as a "poor man's dynamic typing" in core logic.
- Treating `optional` as error handling (it loses diagnostics).
- `variant` "valueless by exception" states: handle carefully in robust code.

Interview angle

Interview Angle

Typical: "When would you use `variant` over inheritance?"
Answer: when the set of types is closed and known; variant gives exhaustive handling and avoids virtual dispatch.

10 Concurrency (Practical): Threads, Locks, Atomics, and Async

10.1 Threads and tasks: choosing the model

Key rules

Rules of Thumb

- Prefer **task-based concurrency** (`async`, thread pools, parallel algorithms) over manual thread management when possible.
- If you use `std::thread`, ensure join/detach correctness:
  - Prefer RAII joiners or `std::jthread` (C++20) for automatic joining.
- Keep shared mutable state minimal; prefer message passing or immutable data.

Common pitfalls

Pitfalls / Gotchas

- Forgetting to join a `std::thread` → `std::terminate`.
- Capturing references into threads when referenced objects die early.
- Oversubscription: too many threads → context switching overhead.

Interview angle

Interview Angle

Expect: "What is a data race?"
Answer: two threads access same memory concurrently, at least one write, without synchronization → undefined behavior.

10.2 Mutexes and lock management

Key rules

Rules of Thumb

- Prefer `std::lock_guard` for simple scoped locking.
- Use `std::unique_lock` when you need deferred locking, unlocking, or condition variables.
- Avoid deadlocks: consistent lock order; use `std::scoped_lock` to lock multiple mutexes safely.

```
std::scoped\_lock lk(m1, m2);
```

Common pitfalls

Pitfalls / Gotchas

- Locking in different orders across code paths.
- Holding locks while calling external code (callbacks, virtual functions) → reentrancy deadlocks.

Interview angle

Interview Angle

Be ready to explain **lock granularity** tradeoff: coarse locks are simple but limit parallelism; fine locks risk deadlocks/overhead.

10.3 Atomics (high-level guidance only)

Key rules

Rules of Thumb

- Use atomics for **simple shared state** (counters, flags) or lock-free structures designed carefully.
- Default memory order: `memory_order_seq_cst` unless you have strong reason and expertise to weaken it.
- Prefer higher-level synchronization (mutex/condvar) for complex invariants.

Common pitfalls

Pitfalls / Gotchas

- Assuming atomic operations alone make compound invariants safe (they do not).
- "Lock-free" is not automatically faster; can be slower and harder to prove correct.

Interview angle

Interview Angle

If asked about memory ordering: explain at a high level (visibility + reordering constraints) and emphasize using defaults unless required.

10.4 `async`, futures, and waiting

Key rules

Rules of Thumb

- Prefer `std::async(std::launch::async, ...)` when you truly want concurrency (otherwise it may defer).
- Use futures to express dependencies; avoid shared mutable state.
- Use condition variables for producer/consumer with a predicate loop.

Common pitfalls

Pitfalls / Gotchas

- Blocking on future in a context that holds a lock → deadlock risk.
- Using condition variables without a predicate loop (spurious wakeups).

Interview angle

Interview Angle

Explain "spurious wakeup": a thread can wake without notification; therefore wait in a loop that checks the predicate.

11 Performance Engineering: Cost Model and Optimization Discipline

11.1 Cost model (what usually dominates)

Key rules

Rules of Thumb

- Big costs: **allocations**, **cache misses**, **branch mispredicts**, **syscalls**, **contention**.
- Prefer contiguous data and linear scans over pointer-chasing.
- Reduce allocations: `reserve`, reuse buffers, prefer `string_view/span` for parameters when safe.
- Virtual dispatch is typically cheap compared to cache misses; measure before refactoring.

Common pitfalls

Pitfalls / Gotchas

- Premature optimization: trading clarity for an unmeasured "gain".
- Micro-optimizing arithmetic while ignoring memory behavior.
- Overusing `shared_ptr` in hot paths (atomic refcount).

Interview angle

Interview Angle

Interviewers love: "How would you optimize this?"
Answer: profile first, identify bottleneck, validate hypothesis, optimize the hot path, re-measure, keep correctness tests.

11.2 Move semantics: benefits and limits

Key rules

Rules of Thumb

- Moves help when:
  - types own heap buffers/resources (strings, vectors),
  - you transfer ownership or reorder containers.
- Moves may not help when:
  - object is small/trivially copyable (copy cheaper than move),
  - the compiler can elide copies anyway (RVO),
  - moving triggers allocations (some types).
- Make move operations `noexcept` when correct to improve container behavior.

Common pitfalls

Pitfalls / Gotchas

- "Sprinkling `std::move` everywhere" harms readability and can pessimize (blocks copy elision, moves from objects still needed).
- Moving from function parameters that are still used later.

Interview angle

Interview Angle

Trick: "Should you `std::move` a local when returning?"
Crisp answer: usually **no**; return by value and let RVO/move happen; explicit `std::move` can inhibit copy elision in some cases.

11.3 constexpr opportunities

Key rules

Rules of Thumb

- Use `constexpr` to shift work from runtime to compile-time when it improves speed and safety.
- Keep compile-time work bounded: do not blow up compile times for tiny runtime gains.

Common pitfalls

Pitfalls / Gotchas

- Excessive template/constexpr metaprogramming that slows builds and harms debuggability.

Interview angle

Interview Angle

Show you can balance tradeoffs: runtime speed vs compile time vs maintainability.

12 Architecture and System Design Guidance

12.1 Layering, dependency direction, and boundaries

Key rules

Rules of Thumb

- Design around **boundaries**: separate policy from mechanism; separate interface from implementation.
- Dependencies should point from **high-level policy** to **abstractions**, not to low-level details.
- Keep components cohesive (single purpose) and loosely coupled (minimal knowledge of others).
- Prefer explicit data flow and ownership across boundaries.

Common pitfalls

Pitfalls / Gotchas

- "God objects" and global singletons: hidden dependencies, hard testing, order-of-init hazards.
- Leaking implementation details in headers (compile-time explosion).

Interview angle

Interview Angle

If asked to improve a design, talk about:

- separating responsibilities,
- clarifying ownership and lifetimes,
- reducing header dependencies,
- choosing error-handling policy at boundaries.

12.2 Logging and observability

Key rules

Rules of Thumb

- Logging is part of the API of a system: define levels, sinks, format, and cost model.
- Avoid logging in tight loops unless sampled/rate-limited.
- Prefer structured logs at boundaries (requests, jobs, transactions) with correlation IDs.

Common pitfalls

Pitfalls / Gotchas

- Using logging as error handling; logs are not control flow.
- Allocations and locks in logger used from hot path → hidden bottlenecks.

Interview angle

Interview Angle

Be ready: "How do you debug production issues?"
Answer: metrics + logs + traces, reproduce with minimal case, add assertions/sanitizers in staging, and validate fixes with tests.

12.3 Design checklists

Modules/components

Design Checklist

- What is the component's responsibility and what is explicitly *not* its responsibility?
- What are the stable interfaces? What can change without rebuilding dependents?
- What is the ownership model for passed objects? (who creates, who destroys)
- What is the error-handling policy across the boundary?
- What are performance constraints and how will you measure them?

Classes

Design Checklist

- Invariant defined? Enforced in constructors? Preserved in public methods?
- Copy/move semantics intended? (value type vs unique resource vs shared)
- Exception safety level documented for mutating operations?
- `const` correctness and thread-safety expectations?
- Representation hidden enough to allow change?

APIs

Design Checklist

- Parameters express nullability and ownership (refs vs pointers vs smart pointers)?
- Returns are lifetime-safe (no dangling views/references)?
- Error channel is unambiguous (exceptions vs status types)?
- Are preconditions cheap and checked (asserts) at the boundary?

13 Pitfalls and Gotchas (High-Value UB Hotspots)

13.1 UB hotspots

Key rules

Rules of Thumb

- Avoid UB by construction: use RAII, avoid raw pointer arithmetic, keep lifetimes explicit.
- Use sanitizers (ASan/UBSan/TSan) regularly in CI and locally.

Common pitfalls

Pitfalls / Gotchas

- Use-after-free / double delete
- Out-of-bounds access
- Dangling references/views
- Iterator invalidation
- Data races (UB)
- Signed overflow (UB)
- Violating strict aliasing (subtle UB)
- Uninitialized reads

Interview angle

Interview Angle

If asked "What is UB and why does it matter?"
Answer: the standard imposes no requirements; compilers optimize assuming UB never happens, so UB can miscompile code and break unpredictably.

13.2 Implicit conversions, narrowing, and overload traps

Key rules

Rules of Thumb

- Prefer explicit conversions: `static_cast` when you mean it.
- Use braces to block narrowing.
- Mark single-argument constructors `explicit` unless implicit conversion is a feature.

Common pitfalls

Pitfalls / Gotchas

- Signed/unsigned mix:

```
std::vector<int> v(10);
for (size_t i = 0; i < v.size()-1; ++i) { /*...*/ } // if v.size()==0 -> wrap
```

- Ambiguous overloads with integer literals and `initializer_list`.

Interview angle

Interview Angle

Trick: "Is signed overflow UB?" Yes (for built-in signed integers). Unsigned overflow wraps modulo 2^N.

13.3 Initialization order fiasco

Key rules

Rules of Thumb

- Avoid non-const globals with dynamic initialization.
- Use function-local statics for safe initialization-on-first-use:

```
Logger& logger() {
  static Logger L{...};
  return L;
}
```

Common pitfalls

Pitfalls / Gotchas

- Global A uses global B in constructor across TUs → B not initialized yet.

Interview angle

Interview Angle

Explain that function-local static initialization is thread-safe since C++11.

13.4 Dangling views and iterator invalidation

Key rules

Rules of Thumb

- Treat `string_view/span/view` as **borrowing** references: document lifetime and keep owners alive.
- After container mutation, assume iterators/references may be invalid unless the container guarantees stability.

Common pitfalls

Pitfalls / Gotchas

- Returning `span` to a local array or vector.
- Keeping an iterator and calling `push_back` on vector then using the iterator.

Interview angle

Interview Angle

Be ready to state invalidation rules for `vector` (realloc invalidates all; erase invalidates after point).

13.5 Concurrency hazards

Key rules

Rules of Thumb

- Synchronize all shared mutable state (mutex/atomics/message passing).
- Prefer immutable data + ownership transfer.

Common pitfalls

Pitfalls / Gotchas

- Sharing `std::string` or `std::vector` across threads without synchronization while mutating.
- Capturing `this` in async tasks that outlive the object.

Interview angle

Interview Angle

Explain "happens-before" at a high level: synchronization creates ordering/visibility guarantees between threads.

14 Interview Drill: Explanations, Trick Questions, and Code Review Checklist

14.1 Explain X like an interviewer

Move vs copy (canonical explanation)

Interview Angle

Copy duplicates resources (two independent owners).
Move transfers resources (steals buffer/handle), leaving source valid but unspecified.
Why it matters: moves enable efficient returns, container growth, and ownership transfer with minimal allocations.

RAII

Interview Angle

Resources are acquired during initialization and released during destruction. This gives deterministic cleanup and makes error handling safe because stack unwinding automatically runs destructors.

ODR

Interview Angle

A program must not contain multiple different definitions of the same entity; templates/inline are special but still must be consistent. ODR violations can cause link errors or subtle runtime bugs.

Vtables

Interview Angle

Runtime polymorphism uses virtual dispatch via an indirection table (vtable). Calls through base pointers/references pick the most-derived override. Layout is implementation-defined; rely on language rules, not layout assumptions.

Templates and concepts

Interview Angle

Templates provide static polymorphism: code is generated for types used, enabling zero-cost abstraction. Concepts add explicit constraints and better error messages, making generic code safer and clearer.

14.2 Common trick questions (crisp answers)

Interview Angle

- **Q:** Does `std::move` move? **A:** No, it casts to an rvalue; the move happens in the move ctor/assign.
- **Q:** Is signed overflow UB? **A:** Yes for built-in signed integers.
- **Q:** Can a destructor throw? **A:** It must not; throwing during unwinding terminates. Design explicit `close()` if failure matters.
- **Q:** Should you return `const T` by value? **A:** Usually no; it blocks moves and adds little value.
- **Q:** When is `shared_ptr` appropriate? **A:** When shared lifetime is a real requirement; otherwise prefer unique ownership and explicit borrowing.
- **Q:** Why `vector` over `list`? **A:** locality and fewer allocations; `list` often slower despite O(1) insert/erase.

14.3 "What would you change in this code?" checklist

Design Checklist

- Ownership explicit? Remove raw owning pointers; use RAII and smart pointers.
- Lifetimes clear? Remove dangling references/views; avoid returning references to locals.
- Invariants stated and enforced? Constructors establish; methods preserve.
- Error handling consistent? Exceptions vs status types; no throwing destructors.
- Performance hazards? Unneeded allocations, poor locality, contention, unnecessary virtual dispatch, repeated rehash/realloc.
- Interface cleanliness? Minimize headers, hide implementation, avoid macros and global state.
- Concurrency correctness? No data races; locks scoped; avoid calling out while holding locks.
- Simplicity? Replace loops with algorithms/ranges where clearer.

1 Compact Reference Snippets

1.1 Minimal "safe defaults" template for a value type

```cpp
struct Vec2 {
  float x{0}, y{0};

  // Rule of 0: no custom destructor/copy/move needed.
  // Provide operations that preserve invariants.
};

inline Vec2 operator+(Vec2 a, Vec2 b) { return {a.x+b.x, a.y+b.y}; }
```

1.2 RAII guard for custom resource

```
class File {
  std::FILE* f{nullptr};
public:
  explicit File(const char* path, const char* mode)
    : f(std::fopen(path, mode)) {
    if (!f) throw std::runtime_error("open failed");
  }
  ~File() noexcept { if (f) std::fclose(f); }

  File(File&& other) noexcept : f(std::exchange(other.f, nullptr)) {}
  File& operator=(File&& other) noexcept {
    if (this != &other) {
      if (f) std::fclose(f);
      f = std::exchange(other.f, nullptr);
    }
    return *this;
  }

  File(const File&) = delete;
  File& operator=(const File&) = delete;
};
```

1.3 Thread-safe initialization-on-first-use

```
Widget& global_widget() {
  static Widget w{/*...*/}; // thread-safe since C++11
  return w;
}
```